

PROFESSIONAL SUMMARY

AI infrastructure engineer with nearly five years of cloud-native AI/HPC experience, focused on production LLM serving since 2024. Leads a Kubernetes-native MaaS control plane for heterogeneous non-NVIDIA accelerators, delivering declarative model-service orchestration, Dynamo Runtime, a P/D Planner, the KV Event Plane, Mooncake-based RDMA KV transfer, and KV-aware routing; also builds BatchFlow, LiteLLM/Higress gateways, autoscaling, failure recovery, and observability. Strong in Go and Python, with Rust engineering experience.

EXPERIENCE

Zhejiang Lab	Aug 2021 - Present
Senior Research Specialist · High-Performance Computing Facility Research Center	2024 - Present
LLM Serving / Model-as-a-Service (MaaS) Platform	
- Designed and productionized a Kubernetes-native model-serving control plane for the MaaS platform, expressing co-located and P/D-disaggregated inference on heterogeneous non-NVIDIA accelerators as declarative services; controllers orchestrate workers, Routers, Services, and Ingresses under unified lifecycle management, failure isolation, and scaling entry points. - Integrated SGLang/vLLM workers with Dynamo DistributedRuntime, connecting worker bootstrap, the KV Event Plane, the Mooncake KV data plane, and a KV-aware Router into one service-discovery, request-routing, KV-reuse, and P/D-transfer path. - Built a production P/D Planner that derives prefill/decode replica targets from request shapes, KV hits, and hardware-specific capacity profiles, then applies auditable scaling decisions with readiness gates, stabilization windows, cooldowns, and rollback protection. - Delivered BatchFlow multi-tenant offline inference, LiteLLM/Higress gateways, and a Rust Router; owned releases, rollbacks, and production diagnosis across queueing, routing, transfer, and service readiness using request IDs, Prometheus metrics, and Kubernetes state.	
Engineering Specialist · Intelligent Supercomputing Research Center	Aug 2021 - 2023
Cloud-Native AI / HPC Platform	
- Customized Kubernetes Device Plugin and resource-management paths for heterogeneous accelerator discovery, node resource reporting, container-runtime validation, device-level allocation, and fault isolation. - Adapted PyTorch workloads, models, and distributed job environments to Linux distributions developed in China and heterogeneous accelerator stacks, establishing the platform foundation for later serving and scheduling work.	

SELECTED PRODUCTION SYSTEMS

Model-Serving Control Plane and Dynamo Runtime	Declarative Serving · Service Discovery · Readiness
- Uses a unified model-service CRD as the lifecycle entry point for co-located or P/D-disaggregated topology, component specifications, runtime policy, and network exposure; controllers continuously orchestrate workers, Routers, Services, and Ingresses and aggregate service status and endpoints, replacing per-model resource assembly with one declarative delivery interface. - Integrates workers with Dynamo DistributedRuntime for service discovery, actual engine-capability publication, and P/D bootstrap; the control plane owns desired state while workers publish roles, concurrency, KV layout, and service endpoints, preventing static configuration from overwriting runtime facts. - Admits traffic only after model health, KV events, the Mooncake transfer channel, P/D bootstrap, and Router registration are ready; on scale-in, stops new admission before draining in-flight requests and unregistering endpoints to protect streaming responses.	
KV Event Plane and Mooncake KV Data Plane	KV-aware Routing · RDMA KV Transfer · Fallback
- Consumed per-DP-rank SGLang/vLLM BlockStored, BlockRemoved, and AllBlocksCleared events into the Event Plane and maintained a Worker/Rank/Storage-Tier prefix radix index. - The Router selects P/D workers from prefix KV overlap, load, accelerator/KV-layout compatibility, and network topology; Mooncake Transfer Engine performs batched KV tensor transfers through the high-performance RDMA fabric, degrading to load-based routing on event anomalies and isolating failed pairs before reselection and retry.	

SELECTED PRODUCTION SYSTEMS (CONTINUED)

Adaptive P/D Planner and Autoscaling Guardrails

Capacity Modeling · P/D Autoscaling · Guardrails

- Aggregates request rate, ISL/OSL, prefill compute tokens and cached tokens, KV hits, queue depth, and running state over rolling windows; selects Capacity Profiles by model, accelerator, quantization, TP/DP, and engine version, using interpolation and conservative fallback for safe capacity estimates.
- Computes prefill/decode token demand, safe per-worker throughput, and independent replica targets, then derives an aggregate target from the bottleneck stage; continuously records auditable scaling decisions with request shape, capacity samples, utilization, and blocking reasons.
- Decouples capacity planning, scaling execution, model-service reconciliation, and Kubernetes scheduling; applies desired > ready pending gates, one-step changes, stabilization windows, cooldowns, stale-decision rejection, and rollback to suppress oscillation and concurrent writes.

KV-aware Router, Multi-Tenant Admission, and Tracing

P/D Routing · Priority Admission · Tracing

- Integrated SGLang HTTP/gRPC P/D routing and selects compatible prefill/decode workers from KV overlap, worker load, and network topology; propagates request IDs, bootstrap/transfer context, and KV telemetry for cross-component diagnosis.
- Implemented P0-P3 priority admission in Rust with global/per-priority semaphores, bounded queues, strict priority, tenant round-robin, and wait timeouts; holds permits until streaming response bodies complete so capacity is not released early.

BatchFlow Multi-Tenant Offline Batch Inference

OpenAI Batch API · Scheduling · Recovery

- Models each batch as a durable, recoverable asynchronous job: the database is the system of record, while schedulers/workers handle transactional claims, sharding, lease renewal, and finalization; Kubernetes Leases, custom_id de-duplication, and shard recovery support multiple replicas.
- Delivered OpenAI Batch API-compatible streaming JSONL, tenant/model concurrency budgets, cancellation, expiry cleanup, output/error writeback, and request-level error isolation to bound offline pressure on shared online model backends.

CORE TECHNOLOGIES

Runtime & Data Plane: Dynamo Runtime / DistributedRuntime, SGLang, vLLM, KV Event Plane, Mooncake Transfer Engine, RDMA KV transfer, P/D disaggregation

Control Plane & Scheduling: Go, Kubernetes Operator, CRD/controller, declarative model-service orchestration, capacity modeling, P/D autoscaling, Device Plugin, Helm, ArgoCD

Routing & Admission: KV-aware routing, Rust, HTTP/gRPC P/D routing, prefix radix index, P0-P3 admission, streaming-request lifecycle

Serving & Observability: Python, OpenAI-compatible API, Batch API, LiteLLM, Higress, Redis, MySQL, Prometheus, QPS/TTFT/TPOT, request tracing

EDUCATION

Arizona State University - M.S. in Computer Science, GPA 3.97 / 4.00, May 2019 - Dec 2020

Arizona State University - B.S. in Computer Science, Aug 2015 - May 2019

PUBLICATION & INTELLECTUAL PROPERTY

Prompt2Length: A Lightweight Model for Predicting Output Length in LLM Inference - third author, IEEE AIAHPC 2025; co-inventor on two granted invention patents and named contributor on two registered software copyrights.

LANGUAGES

Mandarin Chinese (native); English (full professional proficiency).